

## Part01 Questions

### \* Q1: Why does defining a custom constructor suppress the default constructor in C#?

\* Answer: When no constructor is defined, the C# compiler automatically provides a parameterless default constructor. However, once a custom parameterized constructor is explicitly created, the compiler assumes specific initialization rules are required for the object, suppressing the implicit default constructor to enforce those parameters and maintain data integrity.

### \* Q2: How does method overloading improve code readability and reusability?

\* Answer: Method overloading enhances readability by allowing methods that perform logically similar actions to share a unified name (e.g., Sum), eliminating the need for cumbersome names like SumTwoInts or SumThreeInts. It improves reusability by enabling the same core logic to handle various argument counts or data types without duplicating function code.

### \* Q3: What is the purpose of constructor chaining in inheritance?

\* Answer: The primary purpose of constructor chaining in inheritance (using base(...)) is to ensure that the initialization logic of the base class (Parent) executes before the derived class (Child) initializes its own specific members. It prevents code duplication and adheres to the DRY (Don't Repeat Yourself) principle.

### \* Q4: How does new differ from override in method overriding?

\* Answer:

\* **override (Dynamic Binding):** Modifies the virtual table to completely replace the base class implementation. Calling the method through a base-class reference pointing to a derived object executes the overridden method in the derived class.

\* **new (Static Binding / Method Hiding):** Hides the base class method without modifying runtime polymorphism. Calling the method via a base-class reference executes the base method, whereas calling it via a derived-class reference executes the new method.

### \* Q5: Why is ToString() often overridden in custom classes?

\* Answer: By default, ToString() returns the fully qualified type name of the object (e.g., Namespace.ClassName). Overriding it allows developers to provide a meaningful, human-readable string representation of the object's internal state (e.g., (X, Y)), which significantly improves debugging and logging readability.

## Part02 Questions

### \* Q1: What is the difference between class and struct in C#?

\* Answer:

\* **Class:** A Reference Type stored on the Heap. It supports inheritance, garbage collection management, and its default value is null.

\* **Struct:** A Value Type allocated on the Stack (or inline within containing types). It does not support inheritance from other structs/classes and defaults to a value containing zeroed-out fields.

### \* Q2: If inheritance is a relation between classes, clarify other relations between classes.

\* Answer:

\* **Association (Has-A / Weak):** A loose relationship where one class uses another independently without ownership (e.g., Teacher and Student).

\* **Aggregation (Has-A / Shared Ownership):** A whole-part relationship where the child object can exist independently of the parent container (e.g., Department and Teacher).

\* **Composition (Part-Of / Strong Ownership):** A tight whole-part relationship where child objects cannot exist without the parent lifecycle (e.g., Car and Engine).

\* **Dependency (Uses-A):** A temporary relationship where a class receives another object as a parameter within a method execution.

## Part03 Bonus Questions

### \* 1. Self-Study Report:

\* Answer: Object-Oriented Programming (OOP) in C# relies on four fundamental pillars: Encapsulation (protecting data via access modifiers and properties), Abstraction (hiding internal complexity), Inheritance (reusing code across related types), and Polymorphism (enabling dynamic behavior at runtime via virtual/override and interfaces).

### \* 2. What is static and dynamic binding?

\* Answer:

\* **Static Binding (Early Binding):** Resolves method calls at Compile-Time. It applies to regular methods, overloaded methods, and non-virtual method calls using the new keyword, offering faster execution speed.

\* **Dynamic Binding (Late Binding):** Resolves method calls at Run-Time based on the actual object type present in memory. It applies to virtual and overridden methods, enabling dynamic polymorphism.